

Temporal Attention Model Workflow

The temporal attention model, as specified on the right-hand-side of Fig 3, has the following workflow:

1 Entry point

The training process is initiated by the `'__main__'` block in `training.py`. In that block, the user should select a subtype by entering the index. This will set up the relevant subtype flag. The user should also specify the model(s) they wish to run by editing the `model` list. The training process begins by running `training.main()`.

2 Initiate training

```
training.main():
```

- Sets up paths raw and processed data according to the `subtype_flag`.
- Sets up parameters for the training process
- Reads datasets
- Setup tensors and imbalances
- Runs training process for each model, including the baseline models and the paper's own model. During the training process, model evaluation results for the table in Fig 10 are printed out and saved as text files. The training process is handled by `train_model.py`

3 Training and evaluation process

```
train_model.py:
```

For SVM, Random Forest (RF) and Logistic Regression (LR) models:

- Train (data passed into the model) and test (ground truth data to validate model performance)
- The relevant baseline model training procedure is invoked from `train_model.py`, which trains baseline models from existing libraries
- Model predictions are obtained and passed to `validation.evaluate()`, which returns the evaluation statistics for the criteria listed in Fig 10
- Evaluation statistics are printed out and stored to an appropriate text file

For all other supported models:

- A model instance from `models.py` is created (`DaRnnModel` is the paper's own model)

- Training is dispatched to the GPU where available
- The `train_rnn()` procedure is run, which operates a training loop:
 - Sets up relevant data structures and evaluation criterion. Models fulfill Equations (6)-(11) of the paper by setting the encoder to `nn.LSTM` which directly implements these equations
 - Verifies data, where requested
 - For each epoch:
 - Separates data into batches
 - Performs training — Equations (12)-(14) of the paper are implemented during the training process of the `DaRnnModel`, namely in its `temporal_attention` method
 - Makes predictions from the model after training that each batch
 - From the predictions of each batch, retrieve the confusion matrix through `validation.get_confusion_matrix()`, and update running statistics counters to reflect accuracy, precision and MCC up to that batch
 - Calculates epoch-level statistics, and appends to the global lists
 - Obtain model predictions and evaluate criteria shown in Fig 10 using `validation.evaluate()`
 - At regular intervals as specified in the `print_interval` parameter, print out epoch training data
 - Plots a training history graph using the historical statistics
 - Prints out the results of the best epoch, in terms of accuracy scores
 - *(Commented out)* Plots ROC curves
 - *(Commented out)* Plots attention maps as shown in Figures 7-9.

4 Evaluation process in detail

Model evaluation is handled by `validation.evaluate()`, which:

- Obtains the confusion matrix, which contains the number of elements fitting into the following categories, TP (True Positive), FP (False Positive), FN (False Negative) and TN (True Negative), structured as a 2D list `[[TP, FP], [FN, TN]]`
- From confusion matrix, calculates:
 - **Precision:** $\frac{TP}{TP+FP}$ – the proportion of positives that are true
 - **Recall:** $\frac{TP}{TP+FN}$ – the proportion of true values that are reported as positive
 - **F1-score:** $\frac{2 \times \text{precision} \times \text{recall}}{\text{precision} + \text{recall}}$
 - **MCC (Matthew's Correlation Coefficient):** $\frac{TP \times TN - FP \times FN}{\sqrt{(TP+FP) \times (FP+FN) \times (TN+FP) \times (TN+FN)}}$
 - **Accuracy:** $\frac{TP+TN}{TP+FP+FN+TN}$ – the proportion of accurately predicted values

5 Use in Further Data Analysis

The model's predictions are used to generate the mutation probability table in Fig 15, by gathering sequence probabilities by obtaining the scoring of the model.

NB exact code to generate the table is not contained within the code base.